

FastMatMul: Efficient Verifiable Matrix Multiplication via Linear Error-Correcting Codes

Abstract

Verifiable computation systems frequently need to prove large matrix multiplication statements, yet this primitive remains a severe bottleneck for SNARK-based approaches: naïve arithmetization of a $k \times k$ product incurs $O(k^3)$ constraints, and even the classical Freivalds reduction yields $O(k^2)$.

We present FastMatMul, a verifiable computation protocol that reduces the matrix-checking SNARK circuit size to $O(tk + t \log n + E_{cc}(k))$, where t is a small security-parameter-dependent constant and $E_{cc}(k)$ is the cost of the selected code-commitment backend. With a linear-time code-commitment backend, this becomes $O(tk)$. The protocol combines a structured variant of Freivalds’ randomized check with *proximity testing over linear error-correcting codes*: matrices are encoded row-wise and committed via Merkle trees; the verifier detects any inconsistency by querying t random codeword positions and checking lightweight algebraic fold relations. The expensive $O(k^3)$ matrix arithmetic is performed entirely outside the SNARK. We prove that the protocol achieves perfect completeness and computational soundness, relative to certified encoded input commitments and the code-commitment backend, with error $\epsilon_{\text{SNARK}} + \epsilon_{cc} + \epsilon_{\text{bind}} + (k-1)/|\mathbb{F}| + 4(1-\delta)^t$, where δ is the relative distance of the underlying code. The Fiat-Shamir non-interactive variant incurs the standard additional random-oracle loss $\epsilon_{\text{FS}}(q_H)$.

We implement the multiplication-checking layer of FastMatMul in Rust and evaluate it against a direct Freivalds SNARK baseline under the linear-time backend model. Experiments show a 7.5× constraint reduction at $k = 1,024$ and a 15.1× reduction at $k = 2,048$ for this layer. At the largest dimension where both systems complete, FastMatMul achieves up to a 2.95× proving-time speedup; beyond this point the Freivalds baseline exhausts available memory in our benchmark environment, while FastMatMul continues to $k = 4,096$. The proving-time advantage materializes for $k \geq 256$, making the protocol most attractive for large square and near-square matrix multiplication workloads.

ACM Reference Format:

. 2025. FastMatMul: Efficient Verifiable Matrix Multiplication via Linear Error-Correcting Codes. In *Proceedings of ACM Conference on Computer and Communications Security (CCS '25)*. ACM, New York, NY, USA, 15 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '25, Salt Lake City, UT, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2025/10
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large matrix multiplication is a recurring bottleneck in verifiable computation. A prover may wish to convince a verifier that committed matrices A, B, C satisfy $AB = C$ without forcing the verifier, or an underlying SNARK circuit, to recompute the full product. This problem appears across outsourced linear algebra, proof-carrying data pipelines, and verifiable inference for modern neural networks, where projection and attention layers routinely contain matrix products with dimensions $k \geq 1,000$. Naïve SNARK arithmetization of a single $k \times k$ matrix multiplication generates $O(k^3)$ arithmetic constraints, a cost that is impractical even for moderate k .

The $O(k^2)$ barrier. Freivalds’ classical randomized check [8] reduces the problem from $O(k^3)$ to $O(k^2)$: given matrices A, B, C and a random vector r , one verifies $rAB = rC$ instead of recomputing the full product. Several recent systems, including verifiable ML pipelines such as Mystique [18], zkML [4], and the Freivalds-based baseline in [9], adopt this strategy, encoding the vector-matrix products directly as SNARK constraints. The resulting $O(k^2)$ circuit is an improvement, yet it remains expensive: in our optimized baseline, Freivalds already requires 12.6 million R1CS constraints and 117.89 s of proving time at $k = 2,048$, and exhausts available memory in our benchmark environment at $k = 4,096$. Thus the gap between the $O(k^2)$ SNARK cost and practical large-matrix verification remains wide.

Our insight: from computing to checking. We observe that the $O(k^2)$ barrier is *not* inherent to the verification problem—it arises because existing approaches *compute* the vector-matrix products inside the SNARK circuit. Our key idea is to shift the paradigm from “computing in the circuit” to “checking in the circuit”: the prover performs the expensive matrix arithmetic *outside* the SNARK and commits to all intermediate results; the verifier then checks a small number of lightweight algebraic relations that suffice to detect any inconsistency.

The technical engine behind this shift is *proximity testing over linear error-correcting codes*. By encoding each matrix row-wise with a systematic linear code C of relative distance δ , the vector-matrix product $x = rA$ can be verified *column by column* in the encoded domain: the linearity of C ensures that a single inner product (“fold”) of one codeword column suffices to check one coordinate of the encoded product. If the prover cheats on any relation, the committed codewords disagree in at least a δ -fraction of positions, and a random query detects the inconsistency with probability $\geq \delta$. Crucially, the SNARK circuit encodes the *local checks*—fold computations and Merkle path verifications at t queried positions—together with the code-commitment backend checks that bind the intermediate roots to full encoded vectors. The resulting matrix-checking circuit size is $O(tk + t \log n + E_{cc}(k))$, which is *linear* in the matrix dimension when t is a small security-parameter-dependent constant and the selected backend provides $E_{cc}(k) = O(k)$.

Contributions. We present FastMatMul, a verifiable computation protocol for matrix multiplication that realizes this approach. Our contributions are as follows.

- **A novel code-based verification protocol.** We design a three-round public-coin protocol that combines a structured variant of Freivalds’ randomized reduction with proximity testing over linear codes and Merkle-tree commitments. When instantiated with linear-time encodable codes (e.g., expander-based codes [11]) and a linear-time code-commitment backend with $E_{cc}(k) = O(k)$, the resulting SNARK circuit size is $O(tk)$ for fixed security parameter—a significant reduction from the $O(k^2)$ of Freivalds-based approaches. For general constant-rate codes, backend codeword-validity checks contribute an additional $O(k^2)$ term; we analyze this trade-off in Section 4.3. The theorem and the main asymptotic claim are therefore conditional on the chosen code-commitment backend satisfying the stated binding and cost assumptions. The protocol is made non-interactive via the Fiat–Shamir heuristic [7].
- **Rigorous security analysis.** We prove that FastMatMul achieves perfect completeness and computational soundness with error bounded by $\epsilon_{\text{SNARK}} + \epsilon_{cc} + \epsilon_{\text{bind}} + (k-1)/|\mathbb{F}| + 4(1-\delta)^t$, where δ is the code’s relative distance, ϵ_{SNARK} is the SNARK soundness error, ϵ_{cc} is the code-commitment backend’s codeword-binding error, and ϵ_{bind} is the Merkle commitment’s binding error. The Fiat–Shamir non-interactive variant adds the standard random-oracle loss $\epsilon_{\text{FS}}(q_H)$. The theorem is stated relative to certified public input commitments to row-wise encoded matrices and to the selected code-commitment backend, matching the committed-input viewpoint used in code-based proof systems such as Brakedown [11] and Orion [19]. The analysis decomposes the adversary’s strategy into SNARK failure, code-commitment failure, commitment forgery, algebraic evasion, and proximity evasion, and shows each is negligible for standard parameter choices (Section 5).
- **Concrete efficiency gains.** We implement the multiplication-checking layer of FastMatMul in Rust and evaluate it against a direct Freivalds SNARK baseline across matrix dimensions $k \in [2^7, 2^{12}]$ under the linear-time backend model. Key findings (Section 6):
 - *Constraint reduction.* The implemented checking-layer constraints grow as $O(k)$ under the linear-time backend model, versus $O(k^2)$ for Freivalds. The gap reaches $7.5\times$ at $k = 1,024$ (420,197 vs. 3,156,298 constraints) and $15.1\times$ at $k = 2,048$ (835,065 vs. 12,622,154 constraints); Freivalds becomes infeasible beyond $k = 2,048$ in our benchmark environment while the checking-layer implementation scales to at least $k = 4,096$.
 - *Proving-time speedup.* FastMatMul_{Merkle} outperforms Freivalds from $k = 256$ onward, and at the largest jointly feasible dimension ($k = 2,048$) the speedup reaches $2.06\times$ for FastMatMul_{Merkle}, $2.59\times$ for FastMatMul_{KZG}, and $2.95\times$ for FastMatMul_{CPLink}. Beyond $k = 2,048$, Freivalds is infeasible; FastMatMul_{KZG} proves $k = 4,096$ in 117 s.

- *Negligible encoding overhead.* The prover’s off-circuit encoding step takes less than 1 s even at $k = 2^{20}$, confirming that encoding does not introduce a new bottleneck.

Paper organization. Section 2 gives a high-level technical overview. Section 3 recalls the necessary background. Section 4 presents the formal construction and complexity analysis. Section 5 provides the security proof. Section 6 reports our implementation and evaluation. Section 7 surveys related work, and Section 8 concludes with future directions.

2 Technical Overview

We provide a high-level description of our protocol for verifiable matrix multiplication before presenting the formal construction in Section 4. The central challenge is to verify that a claimed product $C = AB$ for matrices $A, B, C \in \mathbb{F}^{k \times k}$ is correct, while keeping the verifier’s work substantially below the $O(k^3)$ cost of recomputing the multiplication. Our approach combines three classical ideas—*randomized verification*, *error-correcting codes*, and *succinct commitment schemes*—in a way that decomposes the verification into components that are each efficient to check inside a SNARK circuit.

2.1 Starting Point: Freivalds’ Algorithm and Its Limitations

The classical starting point for verifying matrix multiplication is Freivalds’ algorithm [8]. Given matrices $A, B, C \in \mathbb{F}^{k \times k}$ and a uniformly random vector $r \in \mathbb{F}^k$, one checks whether $rAB = rC$. If $AB \neq C$, then $D := AB - C \neq 0$, and the check $rD = 0$ fails with probability at least $1 - k/|\mathbb{F}|$ by the Schwartz–Zippel lemma. This reduces a matrix-matrix verification to a vector-matrix computation, collapsing the problem from $O(k^3)$ to $O(k^2)$.

However, even the reduced $O(k^2)$ cost is prohibitive when the check must be performed *inside a SNARK circuit*. In an R1CS or Plonk arithmetization, each field multiplication corresponds to a constraint, so encoding the vector-matrix products rA , rB , and rC directly would still produce $O(k^2)$ constraints. For matrices arising in modern deep-learning inference (where k can reach thousands), this SNARK circuit cost remains impractical.

2.2 From Vectors to Codewords: Leveraging Linear Codes

Our key observation is that the *linearity* of error-correcting codes can be exploited to push the bulk of the computation outside the circuit while retaining verifiability. Concretely, suppose we encode each row of a matrix $M \in \mathbb{F}^{k \times k}$ using a systematic linear code $C : \mathbb{F}^k \rightarrow \mathbb{F}^n$ with relative minimum distance δ . Write $\text{Enc}(M)$ for the matrix whose i -th row is $C(M_{i,\star})$. Because C is $\mathbb{F}$ -linear, taking a linear combination of codewords yields the codeword of the corresponding linear combination of messages:

$$\sum_i r_i \cdot C(M_{i,\star}) = C\left(\sum_i r_i \cdot M_{i,\star}\right). \quad (1)$$

In other words, the “fold” operation $\text{Fold}(\text{Enc}(M), r) := \sum_i r_i \cdot \text{Enc}(M)_{i,\star}$ equals $\text{Enc}(rM)$ exactly. This identity is the engine that

allows us to *check vector-matrix products column-by-column in the encoded domain* without materializing the full vectors.

Commitment via Merkle trees. Each encoded matrix $\text{Enc}(\mathbf{M})$ is committed column-by-column using a Merkle tree: $\text{cm}_M \leftarrow \text{M.CM}(\text{Enc}(\mathbf{M}))$. The verifier starts from certified public roots $(\text{cm}_A, \text{cm}_B, \text{cm}_C)$ that bind the computation inputs and output at the cost of $O(1)$ hash values.

2.3 Proximity-Based Verification via Random Sampling

After the Freivalds reduction, the verification task becomes: given committed encoded matrices $\text{Enc}(\mathbf{A})$, $\text{Enc}(\mathbf{B})$, $\text{Enc}(\mathbf{C})$ and a random challenge r , check that the intermediate vectors

$$\mathbf{x} = \mathbf{rA}, \quad \mathbf{y} = \mathbf{xB}, \quad \mathbf{z} = \mathbf{rC}$$

satisfy $\mathbf{y} = \mathbf{z}$. We now explain how to verify these relations in *sub-linear* time using the encoding.

The prover computes $\mathbf{x}, \mathbf{y}, \mathbf{z}$ outside the SNARK, encodes them, commits to their codewords $(\text{cm}_x, \text{cm}_y, \text{cm}_z)$, and supplies the code-commitment backend material showing that these roots bind to the full encodings $\text{Enc}(\mathbf{x}), \text{Enc}(\mathbf{y}), \text{Enc}(\mathbf{z})$. The verifier then samples an ordered tuple $I = (j_1, \dots, j_t) \leftarrow [n]^t$ of column indices independently with replacement, and for each sampled $j \in I$ requests the j -th column of every committed codeword together with its Merkle authentication path.

At each queried position j , the verifier checks four *local consistency equations*:

- (i) $\text{Enc}(\mathbf{x})_j = \text{Fold}(\text{Enc}(\mathbf{A})^{(j)}, \mathbf{r})$, ensuring $\mathbf{x} = \mathbf{rA}$;
- (ii) $\text{Enc}(\mathbf{y})_j = \text{Fold}(\text{Enc}(\mathbf{B})^{(j)}, \mathbf{x})$, ensuring $\mathbf{y} = \mathbf{xB}$;
- (iii) $\text{Enc}(\mathbf{z})_j = \text{Fold}(\text{Enc}(\mathbf{C})^{(j)}, \mathbf{r})$, ensuring $\mathbf{z} = \mathbf{rC}$;
- (iv) $\text{Enc}(\mathbf{y})_j = \text{Enc}(\mathbf{z})_j$, ensuring the Freivalds check $\mathbf{y} = \mathbf{z}$.

The crucial point is that each of these checks involves only a *single column* of the encoded matrices—an inner product of length k (for the fold operations) plus a scalar comparison. Hence the total verifier work per query is $O(k)$, and for t queries the total is $O(tk)$.

Why does sampling suffice? If any of the underlying vector relations fails—say $\mathbf{y} \neq \mathbf{z}$ —then the codewords $\text{Enc}(\mathbf{y})$ and $\text{Enc}(\mathbf{z})$ are distinct. By the minimum-distance property of C , they disagree in at least a δ -fraction of positions. A uniformly random query lands on a disagreement with probability $\geq \delta$, so t independent queries detect the inconsistency except with probability at most $(1 - \delta)^t$. By a union bound over the four checked relations, the overall proximity-check failure probability is at most $4(1 - \delta)^t$. Setting $t = \lceil (\lambda + 2)/\log_2(1/(1 - \delta)) \rceil$ drives this union-bounded proximity term below $2^{-\lambda}$.

2.4 Putting It Together: The Full Protocol Pipeline

Combining the ingredients above, our protocol FastMatMul proceeds as follows (see Figure 1 for the formal description).

Phase 1: Public input. The verifier is given certified roots $(\text{cm}_A, \text{cm}_B, \text{cm}_C)$ that bind to row-wise encodings of $\mathbf{A}, \mathbf{B}, \mathbf{C}$ under the linear code C .

Phase 2: Challenge. The verifier samples a random field element $r \in \mathbb{F}$ and sends it to the prover. This induces the structured challenge vector $\mathbf{r} = (1, r, r^2, \dots, r^{k-1})$.

Phase 3: Respond. The prover computes the intermediate vectors $\mathbf{x} = \mathbf{rA}$, $\mathbf{y} = \mathbf{xB}$, $\mathbf{z} = \mathbf{rC}$, encodes them, commits to the encodings $(\text{cm}_x, \text{cm}_y, \text{cm}_z)$, and prepares the code-commitment backend proof material for these roots.

Phase 4: Query. The verifier samples $I = (j_1, \dots, j_t) \leftarrow [n]^t$ and sends the sampled indices to the prover. The prover opens the corresponding columns of all six committed codewords along with their Merkle proofs.

Phase 5: Verify (in SNARK). The verifier checks the code-commitment backend relation, Merkle path validity, and the four local consistency equations at each queried index. Crucially, these checks are *arithmetized* into a SNARK relation $\mathcal{R}_{\text{SNARK}}$ (defined formally in Section 4), and the prover produces a succinct proof π attesting that all checks pass. The verifier only needs to verify π and any backend-specific linking material, reducing the core verification cost to that of a single SNARK verification.

Cost summary. The *SNARK circuit size*—the key metric governing prover cost—is $O(t \cdot k + t \log n + E_{\text{cc}}(k))$, where $E_{\text{cc}}(k)$ is the cost of verifying the selected code-commitment backend relation for the intermediate vectors. With a linear-time code-commitment backend, this is linear in k and independent of k^2 or k^3 . The prover additionally performs the native matrix multiplication ($O(k^3)$) and encoding ($O(kn)$) outside the SNARK. The verifier’s core SNARK work is $O(1)$ for pairing-based backends, or $O(|x|)$ for transparent ones; total verification additionally depends on the selected commitment/linking backend. A detailed cost breakdown appears in Table 2 (Section 4.3).

3 Preliminaries

We introduce notation and recall the building blocks used in our construction. Throughout, $\mathbb{F}$ denotes a finite field of size $|\mathbb{F}| = p$ for a prime p , and λ denotes the security parameter. We write $\text{negl}(\lambda)$ for any function that is negligible in λ , and PPT for probabilistic polynomial-time.

3.1 Notation

Matrices are denoted by bold uppercase letters $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$ and vectors by bold lowercase letters $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{F}^k$. For a matrix $\mathbf{M}$, we write $\mathbf{M}_{i,\star}$ for its i -th row and $\mathbf{M}_{\star,j}$ for its j -th column, both viewed as elements of $\mathbb{F}^k$. We write $[n] = \{1, 2, \dots, n\}$ and $I \leftarrow [n]^t$ to denote the uniform sampling of t indices from $[n]$ (with replacement). The expression $r \leftarrow \mathbb{F}$ denotes uniform random sampling from $\mathbb{F}$.

3.2 Linear Error-Correcting Codes

A *linear code* over $\mathbb{F}$ is an $\mathbb{F}$ -linear map $C : \mathbb{F}^k \rightarrow \mathbb{F}^n$ defined by a generator matrix $G \in \mathbb{F}^{k \times n}$, so that $C(\mathbf{m}) = \mathbf{m} \cdot G$ for any message $\mathbf{m} \in \mathbb{F}^k$. The code has *rate* $\rho = k/n$ and *minimum distance* $d = \min_{\mathbf{c} \neq \mathbf{c}'} \Delta(\mathbf{c}, \mathbf{c}')$ over distinct codewords $\mathbf{c}, \mathbf{c}' \in C$, where Δ denotes the Hamming distance. The *relative distance* is $\delta = d/n$. A code is *systematic* if the first k coordinates of $C(\mathbf{m})$ equal $\mathbf{m}$ itself.

We write $\text{Enc}(\mathbf{m}) := C(\mathbf{m})$ for the encoding of a message vector. For a matrix $\mathbf{M} \in \mathbb{F}^{k \times k}$, we define the *row-wise encoding* $\text{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$ as the matrix whose i -th row is $\text{Enc}(\mathbf{M}_{i,\star})$. The j -th column of $\text{Enc}(\mathbf{M})$ is denoted $\text{Enc}(\mathbf{M})^{(j)} \in \mathbb{F}^k$.

The key property we exploit is *linearity*: for any coefficients $r_0, \dots, r_{k-1} \in \mathbb{F}$,

$$\sum_{i=0}^{k-1} r_i \cdot \text{Enc}(\mathbf{M}_{i,\star}) = \text{Enc}\left(\sum_{i=0}^{k-1} r_i \cdot \mathbf{M}_{i,\star}\right). \quad (2)$$

Definition 3.1 (Fold operation). For a column vector $\mathbf{c} = (c_0, \dots, c_{k-1})^\top \in \mathbb{F}^k$ and a coefficient vector $\mathbf{r} = (r_0, \dots, r_{k-1}) \in \mathbb{F}^k$, we define

$$\text{Fold}(\mathbf{c}, \mathbf{r}) := \langle \mathbf{r}, \mathbf{c} \rangle = \sum_{i=0}^{k-1} r_i \cdot c_i.$$

Equivalently, for the j -th column of a row-wise encoded matrix, $\text{Fold}(\text{Enc}(\mathbf{M})^{(j)}, \mathbf{r})$ equals the j -th coordinate of $\text{Enc}(\mathbf{rM})$. This follows directly from the linearity of the code (Equation 2).

Concrete instantiations. Our protocol is compatible with any linear code. Two natural choices are: (i) *Reed–Solomon codes*, where $C(\mathbf{m})$ evaluates the polynomial $\sum_i m_i X^i$ over a domain of size $n > k$, achieving the optimal (MDS) distance $\delta = 1 - k/n + 1/n$; and (ii) *expander-based codes* as used in Brakedown [11], which achieve linear-time encoding $O(n)$ with constant relative distance. We discuss the trade-offs in Section 4.3.

3.3 Merkle Tree Commitment

A *Merkle tree* [15] over a collision-resistant hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ provides a vector commitment scheme with the following algorithms:

- $\text{cm} \leftarrow \text{M.CM}(\mathbf{v})$: Given a vector $\mathbf{v} = (v_1, \dots, v_n)$, construct a binary hash tree with leaves $H(v_1), \dots, H(v_n)$ and output the root hash cm .
- $(v_j, \pi_j) \leftarrow \text{Merkle.Open}(\text{cm}, j)$: Open position j , returning the leaf value v_j and the authentication path π_j (the sequence of sibling hashes from the leaf to the root).
- $b \leftarrow \text{Merkle.Verify}(\pi_j, v_j, \text{cm})$: Verify that v_j is the j -th leaf consistent with root cm ; output $b \in \{0, 1\}$.

The scheme is *position-binding*: under the collision resistance of H , no PPT adversary can produce (j, v, π) and (j, v', π') with $v \neq v'$ such that both verify against the same root cm , except with probability $\epsilon_{\text{bind}}(\lambda) = \text{negl}(\lambda)$.

3.4 Succinct Non-Interactive Arguments of Knowledge

A SNARK (Succinct Non-interactive Argument of Knowledge) for an NP relation $\mathcal{R}$ consists of algorithms (Setup , SNARK.P , SNARK.V):

- $\text{pp} \leftarrow \text{Setup}(1^\lambda, \mathcal{R})$: Generate public parameters.
- $\pi \leftarrow \text{SNARK.P}(\text{pp}, \mathbf{x}, \mathbf{w})$: Given a statement $\mathbf{x}$ and witness $\mathbf{w}$ with $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, produce a proof π .
- $b \leftarrow \text{SNARK.V}(\text{pp}, \mathbf{x}, \pi)$: Verify the proof and output $b \in \{0, 1\}$.

A SNARK satisfies *completeness* (an honest prover always convinces the verifier), *knowledge soundness* (any PPT prover that makes the verifier accept can be used to extract a valid witness),

and *succinctness* (the proof size $|\pi|$ and verifier time are $\text{poly}(\lambda, |\mathbf{x}|)$, independent of the witness size and computation).

Our protocol is modular with respect to the choice of SNARK backend. The key metric is the *circuit size* (number of R1CS constraints or PlonK gates) of the relation $\mathcal{R}_{\text{SNARK}}$ that the backend must prove, as this dominates the prover's cost.

3.5 Freivalds' Algorithm

Freivalds' algorithm [8] is a classical randomized algorithm for verifying matrix multiplication. Given $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$, it samples $\mathbf{r} \leftarrow \mathbb{F}^k$ and checks whether $\mathbf{r}(\mathbf{AB}) = \mathbf{rC}$. The correctness relies on the following:

LEMMA 3.2 (SCHWARTZ–ZIPPEL [17, 21]). Let $P(X_1, \dots, X_m)$ be a non-zero polynomial of total degree d over a finite field $\mathbb{F}$. Then for $r_1, \dots, r_m \leftarrow \mathbb{F}$ chosen uniformly and independently,

$$\Pr[P(r_1, \dots, r_m) = 0] \leq \frac{d}{|\mathbb{F}|}.$$

In our protocol, we use a structured variant: instead of sampling $\mathbf{r} \leftarrow \mathbb{F}^k$ uniformly, we sample a single $r \leftarrow \mathbb{F}$ and set $\mathbf{r} = (1, r, r^2, \dots, r^{k-1})$. If $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$, then for any non-zero column $\mathbf{d}_j$ of $\mathbf{D}$, the inner product $\langle \mathbf{r}, \mathbf{d}_j \rangle = \sum_{i=0}^{k-1} (\mathbf{D})_{i,j} \cdot r^i$ is a non-zero univariate polynomial in r of degree at most $k-1$. By Lemma 3.2, this evaluates to zero with probability at most $(k-1)/|\mathbb{F}|$.

The Vandermonde structure offers a concrete efficiency advantage beyond mere randomness reduction. The challenge r appears in the SNARK public statement $\mathbf{x}$ (Section 4); using a single scalar keeps $|\mathbf{x}| = O(1)$ field elements, independent of k . Had we used a uniformly random $\mathbf{r} \in \mathbb{F}^k$, the statement size would grow to $O(k)$, increasing verifier time for statement-size-sensitive SNARKs (e.g., transparent SNARKs where verifier time is $O(|\mathbf{x}|)$) and defeating the goal of a k -independent verifier cost. The soundness bound $(k-1)/|\mathbb{F}|$ is slightly weaker than the $1/|\mathbb{F}|$ bound of a fully random $\mathbf{r}$, but remains negligible for any $|\mathbb{F}| \geq 2^{256}$ and practical k .

4 Our Construction

In this section we present the formal construction of our verifiable matrix multiplication protocol FastMatMul. We begin by defining the relation chain that reduces the original matrix multiplication statement to a succinct SNARK-friendly relation (Section 4.1), then give the full protocol description (Section 4.2), and conclude with a complexity analysis (Section 4.3). Table 1 summarizes the notation used throughout the paper.

4.1 Relation Reduction Chain

Our construction proceeds through two successive reductions. Each reduction preserves completeness exactly and introduces a bounded soundness error that we analyze in Section 5.

Input-commitment model. Following the committed-oracle viewpoint common in code-based proof systems such as Brakedown [11] and Orion [19], we take the input matrix commitments as certified public inputs. Concretely, $\text{cm}_A, \text{cm}_B, \text{cm}_C$ are assumed to bind to row-wise encodings $\text{Enc}(\mathbf{A}), \text{Enc}(\mathbf{B}), \text{Enc}(\mathbf{C})$ of some matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$. Certification means that every accepting opening of cm_M , for $M \in \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$, is a coordinate or column

Symbol	Description
<i>Parameters</i>	
$\mathbb{F}$	Finite field
k	Matrix dimension ($k \times k$)
n	Codeword length of C
δ	Relative minimum distance of C
$t = I $	Query complexity (number of sampled indices)
λ	Security parameter
<i>Matrices and Vectors</i>	
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	Input/output matrices in $\mathbb{F}^{k \times k}$
$\mathbf{D}$	Error matrix $\mathbf{AB} - \mathbf{C}$
$\mathbf{r}$	Challenge vector $(1, r, r^2, \dots, r^{k-1})$
$\mathbf{x}, \mathbf{y}, \mathbf{z}$	Intermediate vectors: $\mathbf{rA}, \mathbf{xB}, \mathbf{rC}$
<i>Encoding and Commitment</i>	
C	Linear error-correcting code $\mathbb{F}^k \rightarrow \mathbb{F}^n$
$\text{Enc}(\cdot)$	Encoding function (row-wise for matrices)
$\text{Enc}(\mathbf{M})^{(j)}$	j -th column of $\text{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$
$\text{Fold}(\mathbf{c}, \mathbf{r})$	Inner product $\langle \mathbf{r}, \mathbf{c} \rangle$
$\text{M.CM}(\cdot)$	Merkle tree commitment
$\text{Merkle.Open}(\text{cm}, j)$	Merkle tree opening at index j
$\text{Merkle.Verify}(\pi, v, \text{cm})$	Merkle tree verification
$\text{Open.Verify}(\pi, v, \text{cm}, j)$	Position-opening verifier of selected backend
cm_χ	Merkle root for committed object χ
$\pi_{\chi, j}$	Merkle authentication path for χ at position j
σ_v	Code-commitment backend proof/witness for cm_v
<i>Sets and tuples</i>	
$\mathcal{M}$	Matrix set $\{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$
$\mathcal{W}$	Vector set $\{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$
$\mathcal{S}$	Full committed-object set $\mathcal{M} \cup \mathcal{W}$
I	Ordered tuple of queried column indices ($I \in [n]^t$)
<i>SNARK</i>	
$\mathbf{x}$	SNARK public statement (instance)
$\mathbf{w}$	SNARK witness
π	SNARK proof
$\text{SNARK.P}, \text{SNARK.V}$	SNARK prover and verifier
pp	Public parameters

Table 1: Summary of notation.

of $\text{Enc}(\mathbf{M})$ for a fixed matrix $\mathbf{M}$ determined before the verifier samples the Freivalds challenge. This certification may be supplied by the surrounding committed-codeword system, by a public generation process, or by a separate input-opening proof. It is a precondition of the multiplication-checking protocol rather than an online cost charged to FastMatMul. The goal of FastMatMul is to verify the multiplication relation among these committed matrices without arithmetizing the full matrix product. Throughout this section, $\text{M.CM}(\text{Enc}(\mathbf{M}))$ for an encoded matrix means the Merkle root over the length- n sequence of encoded columns $(\text{Enc}(\mathbf{M})^{(1)}, \dots, \text{Enc}(\mathbf{M})^{(n)})$. For an encoded vector, $\text{M.CM}(\text{Enc}(v))$ commits to the length- n sequence of coordinates.

Definition 4.1 (Code-commitment backend). Fix a systematic linear code $C : \mathbb{F}^k \rightarrow \mathbb{F}^n$ and a position-opening layer whose verification predicate is written $\text{Open.Verify}(\pi_j, a, \text{cm}, j)$. For the Merkle instantiation, Open.Verify is exactly Merkle.Verify with the queried

index included in the leaf domain; for KZG or CPLink-style instantiations it is the corresponding opening or linking verifier. A code-commitment backend CodeCom provides a prover relation CodeCom.Prove and a verifier relation CodeCom.Verify for vector commitments. It must satisfy the following two properties.

Completeness. For every $v \in \mathbb{F}^k$, if cm_v commits to $\text{Enc}(v) = C(v)$ under the chosen opening layer, then an honest prover can produce proof material $\sigma_v \leftarrow \text{CodeCom.Prove}(\text{cm}_v, v, \text{Enc}(v))$ accepted by $\text{CodeCom.Verify}(\text{cm}_v, v, \sigma_v)$.

Global codeword binding. The backend is codeword-binding with error $\epsilon_{\text{cc}}(\lambda)$ if, for every PPT adversary $\mathcal{A}$,

$$\Pr[\text{Bad}_{\text{cc}}^{\mathcal{A}}(\lambda) = 1] \leq \epsilon_{\text{cc}}(\lambda).$$

The event $\text{Bad}_{\text{cc}}^{\mathcal{A}}$ occurs if $\mathcal{A}$ outputs $(\text{cm}_v, v, \sigma_v, j, a, \pi_j)$ such that $\text{CodeCom.Verify}(\text{cm}_v, v, \sigma_v) = 1$, $\text{Open.Verify}(\pi_j, a, \text{cm}_v, j) = 1$, and $a \neq \text{Enc}(v)_j$. Thus, except with probability ϵ_{cc} , any vector commitment accepted by CodeCom.Verify is globally linked to the full codeword $\text{Enc}(v)$: every accepting opening at every coordinate is forced to equal the corresponding coordinate of $\text{Enc}(v)$, not merely the sampled coordinates. The separate position-binding error of the opening layer is accounted for by ϵ_{bind} in Theorem 5.1. The cost of verifying $\text{CodeCom.Verify}(\text{cm}_v, v, \sigma_v)$ is denoted $E_{\text{cc}}(k)$.

Rather than introducing a new commitment primitive, FastMatMul is stated relative to this committed-codeword interface, which matches the role played by encoded committed oracles in Brakedown [11] and Orion [19]. Operationally, the backend can be instantiated by a Brakedown/Orion-style code-commitment layer, an in-circuit encoder check tied to the committed root, or a commitment-to-codeword linking proof supplied by the chosen backend.

Backend contract used by the theorem. Theorem 5.1 uses only the following backend facts: (i) the input matrix roots are certified commitments to fixed row-wise codewords before r is sampled; (ii) the intermediate vector roots are sent before I is sampled; (iii) an accepting $\text{CodeCom.Verify}(\text{cm}_v, v, \sigma_v)$ globally links cm_v to $\text{Enc}(v)$ as in Definition 4.1; and (iv) the opening layer is position-binding for every queried coordinate. No part of the proof relies on a new algebraic assumption specific to FastMatMul beyond this committed-codeword interface.

Backend instantiation boundary. The security theorem is compositional: it proves the multiplication-checking protocol relative to any backend satisfying Definition 4.1. An end-to-end system must choose one concrete backend and specify its algorithms, the contents of σ_v , and the constraint/proof-size contribution of CodeCom.Verify . Our implementation evaluates the local fold/opening/linking layer under a linear-time $E_{\text{cc}}(k) = O(k)$ backend model; it does not yet implement or measure CodeCom.Verify independently.

The original relation $\mathcal{R}_{\text{orig}}$. The starting point is the statement that the prover knows matrices whose *encoded* commitments satisfy the multiplication relation. The public instance is $(\text{cm}_A, \text{cm}_B, \text{cm}_C)$ and the witness is $(\mathbf{A}, \mathbf{B}, \mathbf{C})$; the relation $\mathcal{R}_{\text{orig}}$ requires:

$$\text{cm}_A = \text{M.CM}(\text{Enc}(\mathbf{A})) \wedge \text{cm}_B = \text{M.CM}(\text{Enc}(\mathbf{B})) \wedge \text{cm}_C = \text{M.CM}(\text{Enc}(\mathbf{C})), \\ \mathbf{AB} = \mathbf{C}.$$

Thus a matrix commitment binds the verifier to the row-wise code-word matrix rather than to the raw matrix alone. Since C is systematic, the raw matrix is also determined by the first k coordinates of each committed row. Proving membership in $\mathcal{R}_{\text{orig}}$ directly inside a SNARK would require $O(k^3)$ multiplication constraints, which is exactly what we aim to avoid.

Reduction 1: Freivalds' randomization ($\mathcal{R}_{\text{orig}} \rightsquigarrow \mathcal{R}_{\text{Freivalds}}$). Using the verifier's random challenge $r \in \mathbb{F}$, we construct the vector $\mathbf{r} = (1, r, r^2, \dots, r^{k-1})$ and define intermediate vectors $\mathbf{x} = \mathbf{rA}$, $\mathbf{y} = \mathbf{xB}$, $\mathbf{z} = \mathbf{rC}$. Let $\mathcal{S} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{x}, \mathbf{y}, \mathbf{z}\}$ denote the full set of committed objects. The instance is $(\{\text{cm}_\chi\}_{\chi \in \mathcal{S}}, r)$ and the witness is $\{\chi\}_{\chi \in \mathcal{S}}$; the relation $\mathcal{R}_{\text{Freivalds}}$ requires:

$$\begin{aligned} \bigwedge_{\chi \in \mathcal{S}} \text{cm}_\chi &= \text{M.CM}(\text{Enc}(\chi)), \\ \mathbf{rA} &= \mathbf{x} \wedge \mathbf{xB} = \mathbf{y}, \\ \mathbf{rC} &= \mathbf{z} \wedge \mathbf{y} = \mathbf{z}. \end{aligned}$$

By the Schwartz–Zippel lemma, if $\mathbf{AB} \neq \mathbf{C}$, then $\mathbf{y} \neq \mathbf{z}$ with probability at least $1 - (k-1)/|\mathbb{F}|$. Note that while this relation eliminates the $O(k^3)$ matrix product, verifying the vector-matrix products still costs $O(k^2)$ each.

Reduction 2: Proximity lifting ($\mathcal{R}_{\text{Freivalds}} \rightsquigarrow \mathcal{R}_{\text{SNARK}}$). We now encode all matrices and vectors using a systematic $\mathbb{F}$ -linear code $C : \mathbb{F}^k \rightarrow \mathbb{F}^n$ with relative distance δ , and replace the global vector-matrix checks with *local* consistency checks at random positions. Let $I = (j_1, \dots, j_t) \leftarrow \$[n]^t$ be the ordered tuple of queried column indices sampled independently with replacement, and let $\mathcal{M} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$, $\mathcal{W} = \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$ denote the matrix set and vector set respectively.

The *public instance* is $\mathbf{x} = (\{\text{cm}_\chi\}_{\chi \in \mathcal{S}}, r, I)$ and the *witness* is

$$\mathbf{w} = (\mathbf{x}, \mathbf{y}, \mathbf{z}, \sigma_x, \sigma_y, \sigma_z, \{\text{Enc}_\chi^{(j)}, \pi_{\chi,j}\}_{j \in I, \chi \in \mathcal{S}}).$$

The relation $\mathcal{R}_{\text{SNARK}}$ requires $(\mathbf{x}, \mathbf{w})$ to satisfy all of:

$$\begin{aligned} \text{CodeCom.Verify}(\text{cm}_x, \mathbf{x}, \sigma_x) &= 1 \wedge \text{CodeCom.Verify}(\text{cm}_y, \mathbf{y}, \sigma_y) = 1 \\ &\wedge \text{CodeCom.Verify}(\text{cm}_z, \mathbf{z}, \sigma_z) = 1, \quad (\text{CODECOM}) \\ \forall j \in I: \text{Enc}_{x,j} &= \text{Fold}(\text{Enc}_A^{(j)}, \mathbf{r}), \quad (\text{FOLD-A}) \\ \forall j \in I: \text{Enc}_{y,j} &= \text{Fold}(\text{Enc}_B^{(j)}, \mathbf{x}), \quad (\text{FOLD-B}) \\ \forall j \in I: \text{Enc}_{z,j} &= \text{Fold}(\text{Enc}_C^{(j)}, \mathbf{r}), \quad (\text{FOLD-C}) \\ \forall j \in I: \text{Enc}_{y,j} &= \text{Enc}_{z,j}, \quad (\text{EQ}) \\ \forall \chi \in \mathcal{S}, j \in I: \text{Merkle.Verify}(\pi_{\chi,j}, \text{Enc}_\chi^{(j)}, \text{cm}_\chi) &= 1. \quad (\text{PATH}) \end{aligned}$$

Here $\forall j \in I$ ranges over every sampled position in the ordered tuple, including repetitions. For a matrix commitment, $\text{Enc}_\chi^{(j)}$ denotes the full j -th encoded column; for a vector commitment, it denotes the scalar coordinate $\text{Enc}(\chi)_j$. Equation (PATH) is written for the Merkle instantiation used in the transparent variant. In KZG or CPLink-style variants, the same slot is occupied by the selected opening or linking verifier; the security proof uses only position binding of the opening layer and the codeword-binding property of Definition 4.1.

The critical property is that $\mathcal{R}_{\text{SNARK}}$ involves only $O(|I| \cdot k)$ constraints for the fold operations, plus $O(|I| \cdot \log n)$ hash checks for the Merkle paths and the backend code-commitment cost $E_{\text{cc}}(k)$.

Since $|I|$ is a security-parameter-sized constant independent of k , the circuit size is *linear* in k .

Codeword-validity cost. The backend checks in (CODECOM) force the prover to commit to *full* valid vector codewords before the query tuple is sampled. Without this binding step, a cheating prover could choose sampled openings that satisfy the fold checks without committing to any genuine encoding of a fixed vector, defeating the distance-based soundness argument. Analogously, the input matrix commitments $\text{cm}_A, \text{cm}_B, \text{cm}_C$ must be well-formed commitments to row-wise encodings of $\mathbf{A}, \mathbf{B}, \mathbf{C}$. In our model this is part of the public input-commitment interface, not a property re-proved by the multiplication protocol itself. For a general linear code represented by a dense generator matrix $G \in \mathbb{F}^{k \times n}$, the constraint $\text{Enc}_v = v \cdot G$ requires $O(k(n-k))$ inner-product work. For linear-time encodable codes (e.g., expander-based codes [11]) and a code-commitment backend whose verifier follows the encoder circuit, the same validity check can be arithmetized with $O(n)$ constraints, which is $O(k)$ at constant rate. Thus the backend codeword-validity term is linear for the code family used in Table 4, but becomes $O(k^2)$ for dense general constant-rate codes. We discuss the trade-off in Section 4.3.

4.2 Protocol Description

We now give the full specification of FastMatMul. Let $C : \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a systematic linear code with relative distance δ and M.CM denote the Merkle tree commitment. Let $\mathcal{M} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$ be the matrix set, $\mathcal{W} = \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$ the vector set, and $\mathcal{S} = \mathcal{M} \cup \mathcal{W}$. The verifier is given certified input commitments $(\text{cm}_A, \text{cm}_B, \text{cm}_C)$ to $\text{Enc}(\mathbf{A}), \text{Enc}(\mathbf{B}), \text{Enc}(\mathbf{C})$. The protocol FastMatMul operates between a prover $\mathcal{P}$ and verifier $\mathcal{V}$ as shown in Figure 1.

Remark on the SNARK relation. The SNARK proof π attests to the validity of the relation $\mathcal{R}_{\text{SNARK}}$ defined in Section 4.1. The SNARK circuit verifies: (1) constraint (CODECOM), ensuring $\text{cm}_x, \text{cm}_y, \text{cm}_z$ bind, via the selected code-commitment backend, to the full encodings of the same $\mathbf{x}, \mathbf{y}, \mathbf{z}$ used in the fold checks; (2) constraints (FOLD-A)–(EQ), the four fold/equality checks at each $j \in I$; and (3) constraint (PATH), consistency of all Merkle opening proofs with the committed roots. The public statement $\mathbf{x}$ consists of the commitment roots $\{\text{cm}_\chi\}_{\chi \in \mathcal{S}}$, the challenge r , and the query tuple I . The witness $\mathbf{w}$ consists of the intermediate vectors, the backend proofs/witnesses for their codeword commitments, and the opened codeword columns with their authentication paths.

Non-interactive variant. The protocol as described is a three-round public-coin interactive protocol. By applying the Fiat–Shamir heuristic [7] in the random oracle model, it can be made non-interactive using domain-separated transcript hashes:

$$r \leftarrow \text{Hash}(\text{FMM-r}, \text{pp}, \text{cm}_A, \text{cm}_B, \text{cm}_C),$$

and

$$I \leftarrow \text{Sample}_t(\text{Hash}(\text{FMM-I}, \text{pp}, \text{cm}_A, \text{cm}_B, \text{cm}_C, r, \text{cm}_x, \text{cm}_y, \text{cm}_z)).$$

The resulting non-interactive proof consists of the commitment roots, the transcript-derived challenges, and the SNARK proof π ; backend-specific opening, linking, or backend code-commitment material is either carried as part of π or as auxiliary proof data verified by the corresponding backend.

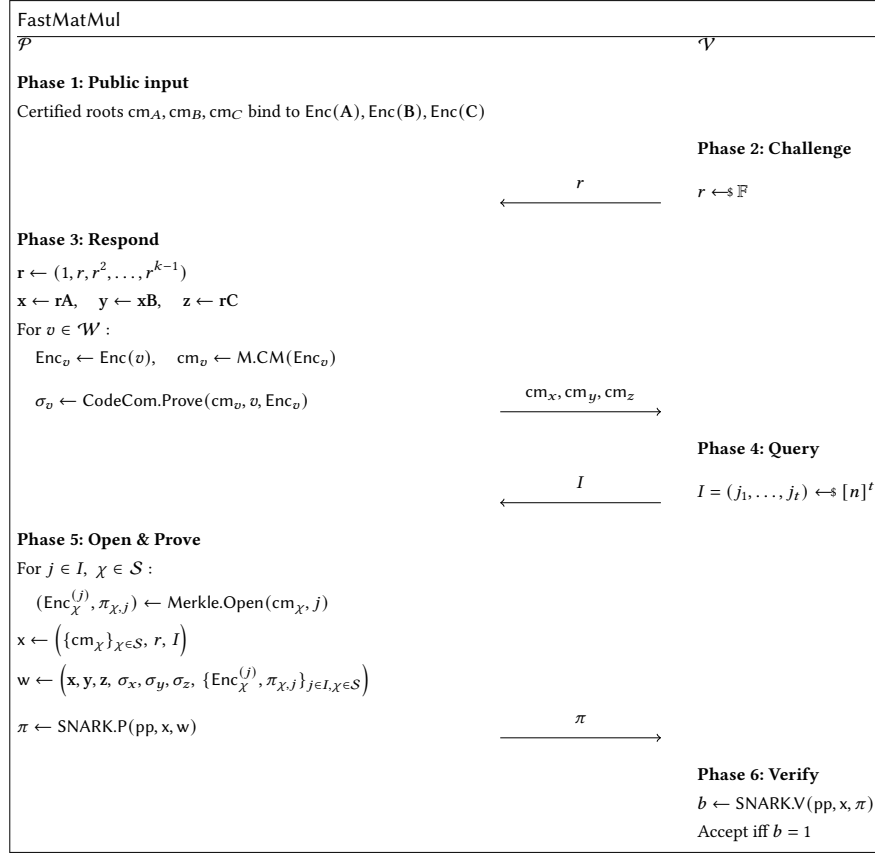


Figure 1: The FastMatMul protocol for verifiable matrix multiplication. $\mathcal{S} = \mathcal{M} \cup \mathcal{W}$ is the full set of committed objects, $t = |I|$ is the query complexity, and $\mathbf{x}, \mathbf{w}$ denote the public statement and witness for the inner SNARK.

Security of the non-interactive variant. A potential concern with the Fiat–Shamir transform for proximity-based protocols is a *grinding attack*: a malicious prover could try many transcript prefixes until the derived challenge r or query tuple I is favorable. The random-oracle theorem accounts for this by the standard $\epsilon_{\text{FS}}(q_H)$ loss in Theorem 5.1. For intuition, consider grinding only the intermediate roots after r is fixed. Suppose the adversary has committed to $(\text{cm}_A, \text{cm}_B, \text{cm}_C)$ (and hence to fixed encoded matrices) and, after receiving r , is searching for $(\text{cm}_x, \text{cm}_y, \text{cm}_z)$ such that the transcript-derived query tuple I avoids the disagreement set D of some fold or equality check. If any of the four checks fails globally, then $|D| \geq \delta n$ by the minimum-distance property. For a single random query $j \in [n]$, the probability of $j \notin D$ is at most $1 - \delta$; for t independent queries it is $(1 - \delta)^t$. Choosing t as in Section 5 makes the union-bounded proximity term $4(1 - \delta)^t$ at most $2^{-\lambda}$. Each trial of the grinding attack produces an independently random I in the random oracle model because the query hash binds the full transcript, so the probability that q grinding attempts yield a favorable I is at most $q \cdot 4(1 - \delta)^t$. For any PPT adversary, q is polynomially bounded in λ , giving a total success probability of $\text{negl}(\lambda)$. Hence the non-interactive protocol retains the soundness bound of Theorem 5.1 in the random oracle model, plus the stated Fiat–Shamir loss.

Zero-knowledge extension. The protocol as stated is a *verifiable computation* protocol and does not conceal the inputs A, B, C from the verifier. Zero-knowledge can be achieved straightforwardly by two modifications: (i) replace the Merkle tree commitments with a *hiding* commitment scheme (e.g., salted hashes or Pedersen vector commitments [16]) so that the committed encoded matrices reveal no information about A, B, C ; and (ii) employ a zero-knowledge SNARK backend for the inner proof π . Masking the intermediate vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$ requires adding fresh random blinding vectors in Phase 3 and including the corresponding decommitment randomness in the SNARK witness. We omit a full ZK construction and proof here; hiding the sampled openings, the backend code-commitment material, and any linking material requires a separate simulation argument.

4.3 Complexity Analysis

We analyze the computational and communication costs of FastMatMul.

Prover computation. The prover’s dominant costs are:

- (i) *Matrix multiplication:* Computing $\mathbf{C} = \mathbf{AB}$ requires $O(k^3)$ field operations (or $O(k^\omega)$ with fast algorithms). This is performed natively outside the SNARK circuit and is amenable to GPU acceleration.

Component	Prover	Verifier
Matrix multiplication	$O(k^3)$	—
Encoding (row-wise)	$O(kn)$	—
Merkle commitment	$O(kn)$	—
SNARK circuit size	$O(tk + t \log n + E_{cc}(k))$	—
Core SNARK verification	—	$O(1)$
Total	$O(k^3 + kn)$	backend-dependent

Table 2: Asymptotic cost comparison. k : matrix dimension; n : codeword length; t : query count; $E_{cc}(k)$: cost of the selected code-commitment backend relation.

- (ii) *Encoding*: $3k$ rows (one per matrix row, three matrices) each encoded into length- n codewords costs $O(kn)$ per matrix with linear-time codes.
- (iii) *Vector computation*: Computing $\mathbf{x}, \mathbf{y}, \mathbf{z}$ costs $O(k^2)$ in total.
- (iv) *SNARK proving*: The circuit size is $O(t \cdot k + t \cdot \log n)$ for the fold and Merkle checks, plus the backend codeword-validity term $E_{cc}(k)$ (see below). The SNARK prover time scales with the circuit size.

Verifier computation. The verifier performs SNARK verification together with any commitment/linking checks required by the chosen backend. For a pairing-based SNARK whose public input consists only of the roots, challenge, and query indices, the core SNARK verification is $O(1)$. For transparent or linking-heavy backends, verification may scale with the public input or linking proof size; the concrete behavior is reported in Section 6.

Communication. The communication depends on the commitment layer. With a fully succinct vector-commitment backend, the proof consists of a constant number of commitment digests plus the SNARK proof, giving $O(\lambda)$ communication. With the Merkle/CPLink implementation evaluated in this paper, the proof also contains authentication, linking, and code-commitment backend material for the queried encoded columns. The sampled-column material scales as $O(tk + t \log n)$ field/hash elements for fixed security parameter; any additional backend material is backend dependent and accounted for in the concrete proof sizes. This trade-off explains the megabyte-scale proof sizes observed in Section 6.4.

Circuit size comparison. Table 2 summarizes the asymptotic costs. Naïve arithmetization of $\mathbf{AB} = \mathbf{C}$ requires $O(k^3)$ constraints. Freivalds' algorithm alone reduces this to $O(k^2)$. Our approach achieves $O(tk + t \log n + E_{cc}(k))$: the first two terms cover the fold and Merkle checks, while $E_{cc}(k)$ denotes the cost of verifying the code-commitment backend relation for the intermediate vector commitments. For dense general codes, $E_{cc}(k) = O(k(n - k))$, which is $O(k^2)$ at constant rate. For linear-time encodable codes of rate ρ and distance δ , $E_{cc}(k) = O(k)$ when the selected code-commitment backend has a linear-time verifier, and setting $t = \lceil (\lambda+2)/\log_2(1/(1-\delta)) \rceil$ gives a total circuit size of $O(k)$ for fixed security parameter, with concrete sizes discussed in Section 6.

5 Security Analysis

We prove that the protocol FastMatMul (Figure 1) constitutes a secure interactive argument for the matrix multiplication relation $\mathcal{R}_{\text{orig}}$ relative to a code-commitment backend and certified encoded input commitments. Soundness and completeness are established in Theorem 5.1 below; knowledge soundness is discussed separately in Remark 2. As discussed in Section 4.1, the public input commitments $\text{cm}_A, \text{cm}_B, \text{cm}_C$ are assumed to bind to row-wise encodings of fixed matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$. This is the committed-input model used by code-based proof systems such as Brakedown [11] and Orion [19]; FastMatMul verifies the multiplication relation among these committed objects. For the intermediate vectors, the SNARK relation invokes the selected code-commitment backend relation $\text{CodeCom.Verify}(\text{cm}_v, v, \sigma_v)$, ensuring that the commitments $\text{cm}_x, \text{cm}_y, \text{cm}_z$ bind to full codewords $\text{Enc}(\mathbf{x}), \text{Enc}(\mathbf{y}), \text{Enc}(\mathbf{z})$ before the query tuple is sampled.

Soundness-critical ordering. The proof uses the following timing invariants. First, the certified input roots $\text{cm}_A, \text{cm}_B, \text{cm}_C$ are fixed before the Freivalds challenge r is sampled. Second, the intermediate roots $\text{cm}_x, \text{cm}_y, \text{cm}_z$ are fixed before the query tuple $I = (j_1, \dots, j_t) \leftarrow \$ [n]^t$ is sampled; conditioned on an accepting backend proof, each such root is globally linked to one full vector codeword independently of the sampled positions. Third, the entries of I are sampled independently with replacement, as defined in Section 3.1. If an implementation samples without replacement, the proximity failure probability only decreases; the theorem states the simpler with-replacement bound.

THEOREM 5.1 (SECURITY OF FASTMATMUL). *The protocol FastMatMul satisfies the following properties:*

- (i) **Perfect completeness.** *If $\mathbf{AB} = \mathbf{C}$, an honest prover always makes the verifier accept.*
- (ii) **Interactive computational soundness.** *For any PPT adversary $\mathcal{P}^*$ given certified public input commitments to $\mathbf{A}, \mathbf{B}, \mathbf{C}$, the probability that the verifier accepts a false committed matrix statement $\mathbf{AB} \neq \mathbf{C}$ satisfies:*

$$\Pr[\mathcal{V} \text{ accepts} \mid \mathbf{AB} \neq \mathbf{C}] \leq \epsilon_{\text{SNARK}}(\lambda) + \epsilon_{\text{cc}}(\lambda) + \epsilon_{\text{bind}}(\lambda) + \frac{k-1}{|\mathbb{F}|} + 4(1-\delta)^t,$$

- Here $t = |I|$ is the query complexity and δ is the relative minimum distance of $\mathbf{C}$. The terms $\epsilon_{\text{SNARK}}, \epsilon_{\text{cc}}$, and ϵ_{bind} denote, respectively, the soundness error of the SNARK backend, the global codeword-binding error of the selected code-commitment backend, and the union-bounded position-binding error of the opening layer over all roots and openings used in the protocol.*
- (iii) **Fiat-Shamir soundness.** *In the random oracle model, the non-interactive variant described in Section 4.2 satisfies the same bound with an additional Fiat-Shamir loss term $\epsilon_{\text{FS}}(q_H)$, where q_H is the adversary's number of random-oracle queries.*

PROOF. We establish the two properties in turn.

Perfect completeness. Suppose the prover is honest and $\mathbf{AB} = \mathbf{C}$. Let $\mathbf{D} := \mathbf{AB} - \mathbf{C} = \mathbf{0}$. For any challenge vector $\mathbf{r}$, we have $\mathbf{rD} = \mathbf{0}$, so the intermediate vectors satisfy:

$$\mathbf{y} = \mathbf{xB} = (\mathbf{rA})\mathbf{B} = \mathbf{r}(\mathbf{AB}) = \mathbf{rC} = \mathbf{z}.$$

The code-commitment backend checks hold by construction: for each $v \in \mathcal{W}$, the honest prover commits to $\text{Enc}(v)$ and provides backend proof material σ_v accepted by CodeCom.Verify . For the fold checks at any index $j \in [n]$, the linearity of C (Definition 3.1) gives:

$$\text{Fold}(\text{Enc}(\mathbf{A})^{(j)}, \mathbf{r}) = \text{Enc}(\mathbf{rA})_j = \text{Enc}_{x,j},$$

and likewise for $\mathbf{B}$ and $\mathbf{C}$. Since $\mathbf{y} = \mathbf{z}$, we also have $\text{Enc}_{y,j} = \text{Enc}_{z,j}$ for all j . All opening or linking checks are valid by honest commitment. Therefore every check passes with probability 1, establishing perfect completeness.

Soundness decomposition. Let Acc denote the event that the verifier accepts, and suppose $\mathbf{AB} \neq \mathbf{C}$. We decompose the adversary's success probability via five bad events:

$$\begin{aligned} \Pr[\text{Acc} \mid \mathbf{AB} \neq \mathbf{C}] &\leq \Pr[\text{Bad}_{\text{SNARK}}] + \Pr[\text{Bad}_{\text{cc}}] + \Pr[\text{Bad}_{\text{bind}}] \\ &\quad + \Pr[\text{Bad}_{\text{rand}} \mid \overline{\text{Bad}_{\text{SNARK}}} \wedge \overline{\text{Bad}_{\text{cc}}} \wedge \overline{\text{Bad}_{\text{bind}}}] \\ &\quad + \Pr[\text{Bad}_{\text{prox}} \mid \overline{\text{Bad}_{\text{SNARK}}} \wedge \overline{\text{Bad}_{\text{cc}}} \wedge \overline{\text{Bad}_{\text{bind}}} \wedge \overline{\text{Bad}_{\text{rand}}}]. \end{aligned}$$

Step 1: SNARK soundness. Define $\text{Bad}_{\text{SNARK}}$ as the event that the verifier accepts a SNARK proof for a false instance of $\mathcal{R}_{\text{SNARK}}$. By soundness of the underlying SNARK,

$$\Pr[\text{Bad}_{\text{SNARK}}] \leq \epsilon_{\text{SNARK}}(\lambda).$$

Conditioned on $\overline{\text{Bad}_{\text{SNARK}}}$, every accepting proof has a witness satisfying the relation in Section 4.1.

Step 2: Code-commitment backend binding. Define Bad_{cc} as the event that, for some intermediate root cm_v and witness vector $v \in \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$, the SNARK accepts $\text{CodeCom.Verify}(\text{cm}_v, v, \sigma_v) = 1$ even though cm_v is not globally linked to the full codeword $\text{Enc}(v)$ under the selected code-commitment backend. Equivalently, there exists a coordinate at which the adversary can produce an accepting opening of cm_v whose value differs from $\text{Enc}(v)$ at that coordinate. By the codeword-binding property of the backend,

$$\Pr[\text{Bad}_{\text{cc}}] \leq \epsilon_{\text{cc}}(\lambda).$$

Conditioned on $\overline{\text{Bad}_{\text{cc}}}$, any accepting proof for the roots $\text{cm}_x, \text{cm}_y, \text{cm}_z$ sent before the query phase can only link those roots to full codewords $\text{Enc}(\mathbf{x}), \text{Enc}(\mathbf{y}), \text{Enc}(\mathbf{z})$ that are independent of the sampled coordinates. In particular, the prover cannot wait for I and then choose vector witnesses whose encodings agree only on the sampled positions.

Step 3: Commitment binding. Define Bad_{bind} as the event that $\mathcal{P}^*$ produces two inconsistent accepting openings for the same root and position under the selected opening layer. For Merkle commitments this is implied by collision resistance of the hash function:

$$\Pr[\text{Bad}_{\text{bind}}] \leq \epsilon_{\text{bind}}(\lambda) = \text{negl}(\lambda).$$

Conditioned on $\overline{\text{Bad}_{\text{bind}}}$, every accepted opening value is uniquely determined by the committed root and queried position. The certified input commitments $\text{cm}_A, \text{cm}_B, \text{cm}_C$ are fixed before the challenge r and determine fixed matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ before r is sampled. Together with the global backend-condition above, the intermediate vector roots determine full committed vector codewords

for any accepting proof, rather than only values at the sampled positions.

Step 4: Randomized reduction. Condition on $\overline{\text{Bad}_{\text{SNARK}}} \wedge \overline{\text{Bad}_{\text{cc}}} \wedge \overline{\text{Bad}_{\text{bind}}}$. The adversary has committed to fixed matrices with $\mathbf{AB} \neq \mathbf{C}$. Let $\mathbf{D} := \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$. Since $\mathbf{D} \neq \mathbf{0}$, there exists at least one column $\mathbf{d}_j$ of $\mathbf{D}$ with $\mathbf{d}_j \neq \mathbf{0}$.

The protocol requires $\mathbf{y} = \mathbf{z}$, i.e., $\mathbf{rD} = \mathbf{0}$. The inner product $\langle \mathbf{r}, \mathbf{d}_j \rangle = \sum_{i=0}^{k-1} (\mathbf{D})_{i,j} \cdot r^i$ defines a non-zero univariate polynomial $P_j(r)$ of degree at most $k-1$ in the random variable r . Since r is chosen after the adversary commits, by the Schwartz–Zippel lemma (Lemma 3.2):

$$\Pr[\text{Bad}_{\text{rand}}] = \Pr_{r \leftarrow \mathbb{F}}[\mathbf{rD} = \mathbf{0}] \leq \Pr_{r \leftarrow \mathbb{F}}[P_j(r) = 0] \leq \frac{k-1}{|\mathbb{F}|}.$$

Step 5: Proximity check. Condition on $\overline{\text{Bad}_{\text{SNARK}}} \wedge \overline{\text{Bad}_{\text{cc}}} \wedge \overline{\text{Bad}_{\text{bind}}} \wedge \overline{\text{Bad}_{\text{rand}}}$. Once r is fixed and $\mathbf{rD} \neq \mathbf{0}$, no choice of intermediate vectors can satisfy all four global relations simultaneously. Therefore the committed vectors violate at least one of:

- (a) $\mathbf{x} \neq \mathbf{rA}$;
- (b) $\mathbf{y} \neq \mathbf{xB}$;
- (c) $\mathbf{z} \neq \mathbf{rC}$;
- (d) $\mathbf{y} \neq \mathbf{z}$.

The SNARK relation $\mathcal{R}_{\text{SNARK}}$ (Section 4.1) includes the backend checks $\text{CodeCom.Verify}(\text{cm}_v, v, \sigma_v) = 1$ for each $v \in \mathcal{W}$. Conditioned on the SNARK, code-commitment, and Merkle binding events above, the opened values under cm_v are therefore coordinates of the genuine codeword $\text{Enc}(v)$. The input matrix commitments are genuine row-wise codewords by the certified input-commitment interface.

Consider case (a): $\mathbf{x} \neq \mathbf{rA}$. Since C is injective, the codewords $\text{Enc}(\mathbf{x})$ and $\text{Enc}(\mathbf{rA})$ are distinct. By the minimum distance property of C :

$$|\{j \in [n] : \text{Enc}(\mathbf{x})_j \neq \text{Enc}(\mathbf{rA})_j\}| \geq \delta \cdot n.$$

By the linearity of C (Definition 3.1), $\text{Fold}(\text{Enc}(\mathbf{A})^{(j)}, \mathbf{r}) = \text{Enc}(\mathbf{rA})_j$. The fold check $\text{Enc}_{x,j} = \text{Fold}(\text{Enc}_A^{(j)}, \mathbf{r})$ fails whenever j falls into the disagreement set, which has size at least δn . A uniformly random query j avoids all disagreements with probability at most $1 - \delta$, so t independent queries all pass with probability at most $(1 - \delta)^t$. The same argument applies symmetrically to cases (b), (c), and (d). Applying the union bound over all four cases:

$$\Pr[\text{Bad}_{\text{prox}}] \leq 4(1 - \delta)^t.$$

Remark on the union bound factor. The factor of 4 is conservative. If the adversary commits to vectors satisfying cases (a)–(c) honestly (i.e., $\mathbf{x} = \mathbf{rA}$, etc.), then $\mathbf{AB} \neq \mathbf{C}$ forces $\mathbf{y} \neq \mathbf{z}$, and only case (d) is active, giving $(1 - \delta)^t$. In general, an adversary deviating on multiple relations is detected by any one of the corresponding checks, so the bound $4(1 - \delta)^t$ covers all strategies without further case analysis.

Combining the bounds. Summing the five error terms:

$$\Pr[\text{Acc} \mid \mathbf{AB} \neq \mathbf{C}] \leq \epsilon_{\text{SNARK}}(\lambda) + \epsilon_{\text{cc}}(\lambda) + \epsilon_{\text{bind}}(\lambda) + \frac{k-1}{|\mathbb{F}|} + 4(1 - \delta)^t.$$

This proves the interactive soundness bound.

Concrete parameters for $\lambda = 128$ bits. Set $|\mathbb{F}| \geq 2^{256}$ so that $(k-1)/|\mathbb{F}| \leq 2^{-128}$ for all practical k . For a λ -bit proximity term, set

$$t = \left\lceil \frac{\lambda + 2}{\log_2(1/(1-\delta))} \right\rceil,$$

which ensures $4(1-\delta)^t \leq 2^{-\lambda}$. For example, if the selected backend certifies $\delta = 1/2$, then strict 128-bit proximity soundness uses $t = 130$. The reported experiments use $t = 128$ for implementation scaling; under $\delta = 1/2$ this gives the slightly weaker proximity term 2^{-126} . For a strict aggregate 128-bit bound over the five interactive error terms, one should allocate the error budget across terms, for example by targeting each term at 2^{-131} and setting $t = \lceil 133/\log_2(1/(1-\delta)) \rceil$. Under standard assumptions, ϵ_{SNARK} , ϵ_{cc} , and ϵ_{bind} are negligible and can be parameterized to the same budget. For the Fiat–Shamir variant, the transcript hashes bind the public input roots, the intermediate roots, the challenge, and the sampled query tuple as described in Section 4.2. The standard random-oracle analysis adds the term $\epsilon_{\text{FS}}(q_H)$ stated above. This completes the proof of Theorem 5.1. $\square$

REMARK 1 (FIAT–SHAMIR SOUNDNESS). *The proof of the main bound in Theorem 5.1 is for the public-coin interactive protocol. The non-interactive variant in Section 4.2 derives both r and I from domain-separated hashes of the full transcript prefix. In the random oracle model, the same proof applies with the standard Fiat–Shamir loss $\epsilon_{\text{FS}}(q_H)$, where q_H is the adversary’s number of random-oracle queries. This term accounts for grinding over transcript prefixes and is negligible for polynomial q_H when the challenge and query hashes have λ -bit security.*

REMARK 2 (KNOWLEDGE SOUNDNESS). *The theorem above is a statement-soundness result relative to certified public input commitments. A full argument of knowledge can be obtained by composing FastMatMul with an extractable input-commitment interface and a knowledge-sound SNARK. In that setting, the input-commitment extractor yields openings of cm_A, cm_B, cm_C to row-wise encoded matrices, while the SNARK extractor yields the intermediate vectors and queried openings used in $\mathcal{R}_{\text{SNARK}}$. The same soundness argument then implies that any prover accepted with non-negligible probability is bound, except with the stated error, to matrices satisfying $\mathbf{AB} = \mathbf{C}$. We treat this as an orthogonal composition issue and focus the main theorem on the multiplication-checking protocol.*

6 Implementation and Evaluation

6.1 Implementation Setup

We implement the multiplication-checking layer of FastMatMul in Rust. We evaluate three commitment and circuit-backend configurations for this layer:

- **FastMatMul_{Merkle}**: a transparent SNARK backend using Merkle-tree commitments implemented with Pedersen hashes. This variant avoids a structured reference string and trades proof size for transparent setup.
- **FastMatMul_{KZG}**: a pairing-based SNARK backend (Groth16) using KZG commitments. This variant requires a structured setup but achieves fast proving at larger circuit sizes due to the efficiency of the pairing-based backend.

- **FastMatMul_{CPLink}**: a hybrid configuration using a Groth16 circuit backend with Merkle-tree commitments and an explicit CP-SNARK linking mechanism. The Groth16 proof component is constant-sized (260 bytes), while the CP-SNARK link and Merkle opening paths constitute the bulk of the proof.

For encoding, we evaluate two configurations:

- **Conditional linear-time backend model** (used in Table 4). We instantiate C with an expander-based code [11] of constant rate and relative distance δ . Table 4 should be read as a conditional linear-time backend configuration: the reported constraint counts cover the implemented fold, opening, and linking circuit terms of the multiplication-checking layer, while the asymptotic claim assumes that the selected CodeCom.Verify contributes $E_{\text{cc}}(k) = O(k)$. The artifact does not yet isolate or independently measure CodeCom.Verify as a standalone component. Dense generator-matrix codeword-validity checks are not included in these rows; Table 7 reports that additional cost for general dense-code instantiations.
- **Constant-rate code** (used in Table 7). We sweep over code rates $\rho \in \{1/2, 1/4\}$ to characterize the dense backend codeword-validity overhead $O(k(n-k)) = O(k^2/\rho)$, which dominates for large k under general dense-code instantiations.

We compare all three variants against an optimized SNARK-in-circuit implementation of Freivalds’ algorithm [8], where the vector-matrix products $\mathbf{rA}$, $\mathbf{xB}$, and $\mathbf{rC}$ are fully arithmetized, incurring $O(k^2)$ R1CS constraints. The optimized Freivalds implementation extends the feasible range to $\log k = 11$ ($k = 2,048$), compared to the earlier implementation which was limited to $\log k = 10$. All experiments use matrices over a 256-bit prime field with k ranging from 2^7 to 2^{12} .

Security and code parameters. Table 3 summarizes the parameters recorded by the benchmark artifacts. The main matrix-multiplication experiments use rate $\rho = 1/2$, codeword length $n = 2k$, and $t = 128$ sampled positions. These rows are scaling measurements for the implemented checking layer, not a complete deployment parameter certificate. The security theorem in Section 5 uses $t = \lceil (\lambda + 2)/\log_2(1/(1-\delta)) \rceil$ so that the four-way proximity union bound is at most $2^{-\lambda}$. For an ideal rate-1/2 distance calculation with $\delta = 1/2$, the reported $t = 128$ rows give a proximity term $4(1/2)^{128} = 2^{-126}$; a strict 128-bit proximity term would use $t = 130$. If the selected expander-code backend certifies a smaller δ , then t must be increased according to the displayed formula. Moving from $t = 128$ to $t = 130$ changes the t -linear fold/opening terms by 1.6%, so it does not change the observed scaling trend.

Timing scope. The proving-time columns in Table 4 report online proving time and exclude one-time setup/key-generation time. For the FastMatMul artifact, total proving time includes matrix commitment, vector commitment, circuit proving, and CPLink/KZG-style linking when present; native matrix multiplication and setup are recorded separately. For the Freivalds baseline, proving time likewise excludes native matrix computation and setup. The experiment logs record setup separately: for example, the CPLink configuration ranges from 7.27 s at $\log k = 7$ to 171.40 s at $\log k = 12$, while

Parameter	Value in main experiments
Field size	256-bit prime field
Matrix dimensions	$k \in \{2^7, \dots, 2^{12}\}$
Code rate	$\rho = 1/2$
Codeword length	$n = 2k$
Sampled positions in reported runs	$t = 128$
Backend model	Conditional linear-time code-commitment backend
Certified distance used by theorem	Backend-specific δ
Strict 128-bit proximity target	$t = \lceil (\lambda + 2)/\log_2(1/(1 - \delta)) \rceil$; $t = 130$ when $\delta = 1/2$

Table 3: Concrete parameters used in the main experiments. The reported rows use $t = 128$ to compare implementation scaling. A deployment should choose t from the theorem using the certified distance δ of the selected code-commitment backend.

the Freivalds setup ranges from 5.45 s at $\log k = 7$ to 1303.91 s at $\log k = 11$. Native matrix computation, matrix commitment, vector commitment, circuit proving, linking, and verification are recorded as separate columns in the artifact CSVs, but the current artifact does not yet log peak memory or timing variance.

Artifact limitations. The current benchmark artifact is sufficient to support the scaling claims for the implemented multiplication-checking layer in Table 4, but it does not yet provide a standalone CodeCom.Verify implementation, a per-component constraint breakdown, peak resident memory, or repeated-run statistics. Accordingly, we report Freivalds’ $\log k = 12$ failure as an observed out-of-memory outcome in the benchmark environment, not as a fitted memory scaling law. These missing measurements are orthogonal to the soundness theorem but are important for a complete systems evaluation.

6.2 Constraint Count Comparison

Table 4 and Figure 2 report constraint counts for all four configurations. The asymptotic gap between the two approaches is clearly visible. For Freivalds, constraints grow by a factor of approximately $4\times$ with each doubling of k —a signature of $O(k^2)$ scaling—while all three FastMatMul variants grow by approximately $2\times$, confirming $O(k)$ scaling of the implemented checking layer under the conditional linear-time backend model.

The crossover in absolute constraint count occurs at $k = 256$ ($\log k = 8$): below this threshold, Freivalds’ $O(k^2)$ constant is sufficiently small that its circuit is marginally smaller (49,610 vs. 53,727 for FastMatMul_{Merkle} at $\log k = 7$); above it, all FastMatMul variants are consistently more compact. At $k = 1,024$ ($\log k = 10$), FastMatMul_{Merkle} uses 420,197 constraints versus 3,156,298 for Freivalds—a $7.5\times$ reduction. At $k = 2,048$ ($\log k = 11$), the gap widens to $15.1\times$: FastMatMul_{CPLink} requires only 835,065 constraints versus 12,622,154 for Freivalds. Beyond $\log k = 11$, Freivalds exhausts available memory in the benchmark environment, while all FastMatMul variants scale to at least $k = 4,096$ ($\log k = 12$)

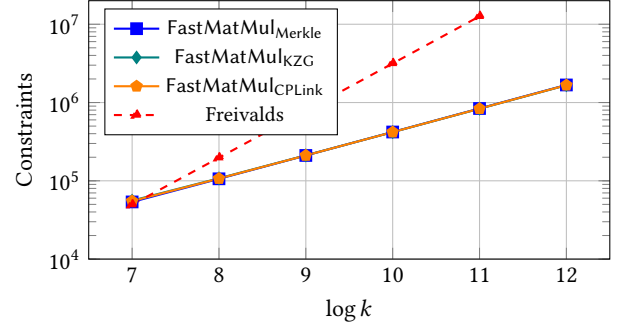


Figure 2: Constraint count comparison on a log scale. All three FastMatMul variants exhibit $O(k)$ checking-layer growth under the conditional linear-time code-commitment backend model (slope ≈ 1), while Freivalds exhibits $O(k^2)$ (slope ≈ 2). Freivalds does not complete at $\log k = 12$ in the benchmark environment.

with under 1.68 M constraints. This confirms that FastMatMul extends the practical range of SNARK-based matrix multiplication verification by at least one order of magnitude in matrix dimension.

6.3 Proving Time

Figure 3 and Table 4 show the prove-time comparison across all four configurations. The crossover point—at which FastMatMul variants begin to outperform Freivalds—varies by variant. FastMatMul_{Merkle} achieves crossover at $\log k = 8$ (2.32 s vs. 2.36 s), while FastMatMul_{KZG} and FastMatMul_{CPLink} cross over between $\log k = 9$ and $\log k = 10$.

At $\log k = 7$, Freivalds is faster than all FastMatMul variants (0.66 s vs. 1.07 s for FastMatMul_{Merkle}, 2.74 s for FastMatMul_{KZG}, and 1.98 s for FastMatMul_{CPLink}). As k grows, the $O(k^2)$ circuit cost of Freivalds dominates: at $\log k = 10$, FastMatMul_{Merkle} achieves $1.71\times$ speedup (17.48 s vs. 29.89 s); at $\log k = 11$, the speedup reaches $2.06\times$ for FastMatMul_{Merkle} (57.18 s vs. 117.89 s), $2.59\times$ for FastMatMul_{KZG} (45.47 s), and $2.95\times$ for FastMatMul_{CPLink} (40.03 s). Beyond $\log k = 11$, Freivalds is infeasible in the benchmark environment due to memory exhaustion, while all FastMatMul variants complete $\log k = 12$: FastMatMul_{KZG} in 117.39 s, FastMatMul_{Merkle} in 193.50 s, and FastMatMul_{CPLink} in 200.19 s.

We note that at small k ($\log k = 7$), Freivalds proves faster due to two factors: (i) the per-constraint overhead of FastMatMul includes Merkle-path hashing and CP-SNARK linking, which contribute fixed overhead per query; and (ii) the $O(k^2)$ constant in Freivalds’ circuit is sufficiently small at this scale. For $k \geq 256$, these constants are progressively dominated by the circuit size difference. Among the FastMatMul variants, FastMatMul_{KZG} achieves the fastest proving at larger k (45.47 s vs. 57.18 s for FastMatMul_{Merkle} and 40.03 s for FastMatMul_{CPLink} at $\log k = 11$; 117.39 s vs. 193.50 s and 200.19 s at $\log k = 12$), reflecting the efficiency of the pairing-based backend for larger circuits. FastMatMul_{CPLink} achieves competitive circuit-level proving (its Groth16 circuit proof time is as low as 0.81–16.15 s), but its total prove time at $\log k = 12$ is dominated by the matrix commitment phase (174.45 s out of 200.19 s).

log k	Constraints				Prove Time (s)				Verify Time (s)			
	Merkle	KZG	CPLink	Freivalds	Merkle	KZG	CPLink	Freivalds	Merkle	KZG	CPLink	Freivalds
7	53,727	55,828	55,059	49,610	1.07	2.74	1.98	0.66	0.06	0.23	0.49	0.01
8	106,081	107,546	106,905	197,194	2.32	4.61	4.47	2.36	0.10	0.27	1.70	0.04
9	210,787	211,370	210,985	788,810	5.84	8.53	11.26	7.99	0.19	0.35	2.43	0.14
10	420,197	419,012	419,139	3,156,298	17.48	17.95	23.67	29.89	0.33	0.49	1.14	0.60
11	839,015	833,914	835,065	12,622,154	57.18	45.47	40.03	117.89	0.64	0.78	1.83	2.36
12	1,676,649	1,664,106	1,667,305	—	193.50	117.39	200.19	—	1.38	1.37	2.83	—

Table 4: Performance comparison of FastMatMul variants and the Freivalds baseline. All three FastMatMul variants exhibit $O(k)$ checking-layer constraint growth under the conditional linear-time code-commitment backend model, versus $O(k^2)$ for Freivalds. FastMatMul_{Merkle} uses a transparent SNARK backend with Merkle-tree commitments implemented with Pedersen hashes; FastMatMul_{KZG} uses a pairing-based (Groth16) backend with KZG commitments; FastMatMul_{CPLink} uses a Groth16 circuit backend with Merkle-tree commitments and CP-SNARK linking. The table does not isolate or independently measure a standalone CodeCom.Verify component. Freivalds uses an optimized implementation. “—” indicates the experiment did not complete in the benchmark environment because it exhausted available memory; peak memory was not logged by the current artifact. Proof sizes for FastMatMul variants range from 1.70–48.79 MB (see Section 6.4).

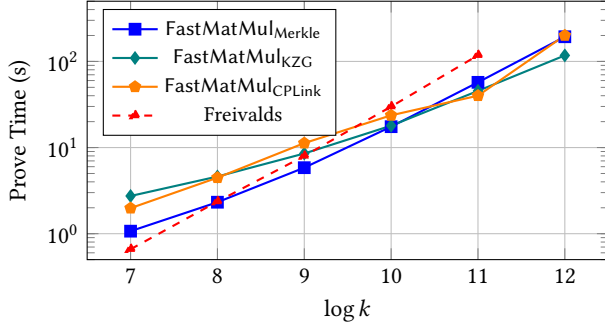


Figure 3: Proving time comparison on a log scale. FastMatMul_{Merkle} becomes faster than Freivalds at $\log k = 8$; FastMatMul_{KZG} and FastMatMul_{CPLink} cross over between $\log k = 9$ and $\log k = 10$. At $\log k = 11$, the speedup reaches $2.06\times$ – $2.95\times$. Freivalds does not complete at $\log k = 12$ in the benchmark environment.

6.4 Verification Time and Proof Size

Table 4 reports verification times for all four configurations. FastMatMul_{Merkle} achieves the fastest verification among the FastMatMul variants, growing from 0.06 s at $\log k = 7$ to 1.38 s at $\log k = 12$, approximately doubling with each doubling of k . FastMatMul_{KZG} exhibits a similar trend (0.23–1.37 s). FastMatMul_{CPLink} incurs higher verification overhead (0.49–2.83 s), dominated by its CP-SNARK link verification component (0.47–2.52 s). This growth reflects the cost of linking the t queried encoded columns to the committed objects; the linked column material scales as $O(tk)$ for fixed security parameter.

The optimized Freivalds baseline exhibits verification times that grow from 0.01 s at $\log k = 7$ to 2.36 s at $\log k = 11$, scaling roughly as $O(k^2)$ with the circuit size. At $\log k = 11$, FastMatMul_{Merkle} verification (0.64 s) is $3.7\times$ faster than Freivalds (2.36 s), demonstrating that the $O(k)$ circuit structure of FastMatMul yields a verification advantage at sufficiently large k .

Proof sizes for all FastMatMul variants are comparable: FastMatMul_{Merkle} grows from 1.70 MB ($\log k = 7$) to 48.79 MB ($\log k = 12$), FastMatMul_{KZG}

log k	Merkle paths (KB)	CPLink proof (MB)	Total (MB)
7	160	1.57	1.72
8	180	3.07	3.24
9	200	6.07	6.26
10	220	12.07	12.28
11	240	24.07	24.30
12	260	48.07	48.32

Table 5: FastMatMul_{CPLink} proof-size breakdown from the benchmark artifact. The Groth16 component is 260 bytes in every row and is omitted from the table because it is negligible relative to the linking material.

from 1.76 to 48.36 MB, and FastMatMul_{CPLink} from 1.72 to 48.32 MB—all roughly doubling with each unit increase in $\log k$, consistent with the $O(k)$ witness size dominated by the opened codeword columns and their Merkle authentication paths. For FastMatMul_{CPLink}, the proof comprises a constant-sized Groth16 component (260 bytes), Merkle opening paths (160–260 KB), and a CP-SNARK link proof (1.57–48.07 MB); the CP-SNARK link constitutes over 90% of the total proof size.

We remark that the proof size and verification time of FastMatMul can be further improved by replacing the Merkle-based commitment with a fully algebraic vector commitment scheme (e.g., KZG [12] applied directly to the commitment layer), which could reduce proof size to $O(\lambda)$ and verification to $O(1)$ at the cost of a structured setup. We leave this optimization as future engineering work.

6.5 Encoding Cost Analysis

Table 7 reports the precomputation and encoding times under general constant-rate codes for $\log k \in [10, 20]$ and rates $\rho \in \{1/2, 1/4\}$. Two observations are noteworthy.

Encoding is fast. Even at $\log k = 20$ ($k = 2^{20} \approx 10^6$), encoding completes in 0.623 s (rate 1/2) and 0.680 s (rate 1/4), and precomputation takes at most 0.220 s. These costs are negligible relative to the native $O(k^3)$ matrix multiplication, confirming that encoding does not introduce a new bottleneck in the prover’s pipeline.

Variant	Proof object
FastMatMul _{Merkle}	Commitment roots, sampled encoded columns, Merkle authentication paths, code-commitment backend material, and the transparent SNARK proof.
FastMatMul _{KZG}	Commitment roots, sampled encoded columns, KZG/Groth16 proof material, code-commitment backend material, and openings required by the backend.
FastMatMul _{CPLink}	Commitment roots, sampled encoded columns, Merkle authentication paths, a constant-sized Groth16 proof, and a CP-SNARK link proof tying the opened columns to the committed objects.

Table 6: Proof-object components by backend. The measured proof size is dominated by sampled-column opening and linking material in the evaluated Merkle/CPLink-style configurations.

$\log k$	ρ	Precomp. (s)	Encoding (s)	Constraints
10	1/2	0.000	0.001	5,134
10	1/4	0.000	0.001	9,244
12	1/2	0.000	0.002	20,536
12	1/4	0.001	0.003	36,974
14	1/2	0.002	0.008	82,140
14	1/4	0.005	0.010	147,896
16	1/2	0.007	0.035	328,558
16	1/4	0.012	0.040	591,578
18	1/2	0.024	0.126	1,314,228
18	1/4	0.049	0.152	2,366,310
20	1/2	0.104	0.623	5,256,908
20	1/4	0.220	0.680	9,465,240

Table 7: Precomputation and dense codeword-validity cost under general constant-rate codes. ρ : code rate; Constraint: number of dense backend codeword-validity R1CS constraints (additional to the fold/Merkle constraints in Table 4).

Rate vs. constraint trade-off. The dense codeword-validity constraints—which contribute $O(k(n-k)) = O(k^2(1-\rho)/\rho)$ to the circuit under dense generator-matrix checks—grow roughly as 2 $\times$ when the rate is halved, as expected. At $\log k = 16$, rate 1/2 yields 328,558 backend constraints while rate 1/4 yields 591,578 constraints (1.8 $\times$ more). For the Merkle-hash component (fold and path checks), the constraint count is $O(tk)$ and appears in the FastMatMul columns of Table 4; the constraints in Table 7 represent an *additional* backend codeword-validity overhead incurred when using general dense codes instead of linear-time code-commitment backends.

When the backend codeword-validity cost is acceptable (i.e., when k^2 fits within the SNARK’s capacity), rate 1/2 represents a good default, offering a balanced trade-off between code distance δ (which determines the soundness error per query) and backend overhead. For applications demanding a higher soundness level per query (larger δ), a lower rate with more queries may be preferable.

6.6 Summary

Table 4 and Table 7 together establish the following conclusions about FastMatMul.

Constraint efficiency. All three FastMatMul variants achieve $O(k)$ checking-layer circuit growth under the conditional linear-time code-commitment backend model, versus $O(k^2)$ for Freivalds-based SNARK arithmetization. The measured checking-layer constraint advantage reaches 7.5 $\times$ at $k = 1,024$ ($\log k = 10$) and 15.1 $\times$ at $k = 2,048$ ($\log k = 11$). Even with an optimized Freivalds implementation that extends to $\log k = 11$, the baseline does not complete at $\log k = 12$ in our benchmark environment, while all FastMatMul variants scale to at least $k = 4,096$.

Proving time. FastMatMul_{Merkle} outperforms Freivalds from $\log k = 8$ onward (2.32 s vs. 2.36 s); FastMatMul_{KZG} and FastMatMul_{CPLink} cross over between $\log k = 9$ and $\log k = 10$. At $\log k = 11$, speedups reach 2.06 $\times$ (FastMatMul_{Merkle}), 2.59 $\times$ (FastMatMul_{KZG}), and 2.95 $\times$ (FastMatMul_{CPLink}). For $k = 128$ ($\log k = 7$), Freivalds proves faster due to its lower per-constraint overhead at small circuit sizes. Among the FastMatMul variants, FastMatMul_{KZG} achieves the fastest proving at larger k , reaching 1.65 $\times$ speedup over FastMatMul_{Merkle} at $\log k = 12$.

Verification and proof size. FastMatMul_{Merkle} achieves the lowest verification overhead among FastMatMul variants (0.06–1.38 s), followed by FastMatMul_{KZG} (0.23–1.37 s) and FastMatMul_{CPLink} (0.49–2.83 s). Freivalds verification grows from 0.01 s ($\log k = 7$) to 2.36 s ($\log k = 11$), scaling with its $O(k^2)$ circuit size. At $\log k = 11$, FastMatMul_{Merkle} verification (0.64 s) is 3.7 $\times$ faster than Freivalds. Proof sizes for all FastMatMul variants range from 1.70–48.79 MB and can be further improved by applying algebraic commitment schemes directly to the commitment layer.

Deployment regime. The crossover occurs at dimensions already common in large linear-algebra workloads: FastMatMul_{Merkle} becomes faster than Freivalds at $k = 256$, and all FastMatMul variants provide a large constraint advantage by $k = 1,024$. This makes the protocol most attractive for large square or near-square matrix products when the application can supply the required code-commitment backend and tolerate the measured opening/linking material.

Practical recommendation. For verifiable matrix-multiplication workloads with $k \geq 256$, FastMatMul is attractive under the certified-input and linear-time-backend assumptions. FastMatMul_{KZG} offers the best proving time, FastMatMul_{Merkle} provides a transparent setup without a structured reference string, and FastMatMul_{CPLink} achieves the highest speedup over Freivalds at the largest jointly feasible dimension (2.95 $\times$ at $\log k = 11$) with a compact Groth16 circuit proof. Their combination of linear circuit growth and competitive proving time makes FastMatMul most attractive in settings where the prover bears the native matrix-arithmetic cost and the application can tolerate backend-dependent opening and linking material.

7 Related Work

We position our work within four lines of research: verifiable ML inference, dedicated matrix multiplication verification, code-based proof systems, and the sumcheck approach.

Verifiable ML inference. SafetyNets [9] was among the first to propose interactive proofs for deep neural networks, employing the sumcheck protocol to verify matrix multiplications layer by layer. zkCNN [14] extended this to convolutional layers by reducing convolution to polynomial multiplication and leveraging the GKR protocol [10]. vCNN [13] further optimized the verification of quantized convolution operations. Mystique [18] and zkML [4] have explored practical zero-knowledge proof systems for general ML inference, including Transformer architectures. In all of these systems, matrix multiplication is the dominant cost center. Our protocol is complementary: it provides a drop-in replacement for the matrix multiplication verification subroutine that reduces the SNARK circuit size from $O(k^2)$ to $O(tk + t \log n + E_{cc}(k))$, and to $O(tk)$ when the code-commitment backend verifier is linear. Composing FastMatMul across the layers of a neural network is a natural extension that we outline in Section 8.

Dedicated matrix multiplication verification. zkMatrix [5] proposed a batched short-proof protocol for committed matrix multiplication using customized polynomial commitments, achieving proof sizes of $O(\sqrt{k})$ group elements. DualMatrix [6] combined polynomial evaluation and inner-product arguments to reduce the prover cost, though the prover still performs $O(k^2)$ work inside the proof system. zkVC [20] introduced Constraint-Reduced Polynomial Circuits (CRPC) and Prefix-Sum Query (PSQ) techniques to reduce the R1CS constraint count, reporting a 12 $\times$ prover speedup and demonstrating applicability to Transformer inference. Bennett et al. [3] explored the use of coding theory for matrix multiplication verification in an information-theoretic setting, establishing theoretical connections between code distance and verification complexity. Our work shares the coding-theoretic perspective of [3] but differs in two key respects: (i) we operate in the computational setting with committed-codeword openings and SNARKs, yielding a *non-interactive* protocol with succinct proofs; and (ii) we provide a concrete implementation with experimental evaluation, demonstrating practical gains over the Freivalds baseline. Relative to zkMatrix, DualMatrix, and zkVC, our construction is modular in the commitment layer: the transparent instantiation uses hash-based openings, while the KZG and CPLink configurations expose the trade-off between algebraic commitments, linking material, and setup assumptions.

Code-based proof systems. Ligerio [1] pioneered the use of interleaved Reed–Solomon codes to construct transparent zero-knowledge proofs with $O(\sqrt{n})$ proof size. Brakedown [11] introduced expander-based linear codes as a drop-in replacement for polynomial commitments, achieving linear-time prover complexity for general-purpose R1CS. Orion [19] combined code-based commitments with the sumcheck protocol. FRI [2] (Fast Reed–Solomon Interactive Oracle Proofs of Proximity) established the foundation for proximity testing of Reed–Solomon codes, underpinning the STARK family of proof systems. Our work draws on the code-based commitment philosophy of Brakedown but applies it to the *structured* setting

of matrix multiplication, where the linearity of encoding interacts directly with the Freivalds reduction to yield a circuit-size improvement beyond what a generic code-based SNARK achieves. Specifically, while Brakedown provides a general $O(n)$ -prover SNARK for any circuit of size n , naively feeding the $O(k^2)$ Freivalds circuit into Brakedown still yields an $O(k^2)$ prover. FastMatMul’s contribution is to *restructure the problem itself*—via the relation chain $\mathcal{R}_{\text{orig}} \rightarrow \mathcal{R}_{\text{Freivalds}} \rightarrow \mathcal{R}_{\text{SNARK}}$ —so that the circuit presented to the SNARK is only $O(tk + t \log n + E_{cc}(k))$.

Sumcheck-based approaches. The sumcheck protocol [10] is a powerful interactive proof technique for verifying the evaluation of multivariate polynomials. When applied to matrix multiplication, it reduces the verifier’s work to evaluating multilinear extensions of the input matrices at a random point, which requires $O(k^2)$ field operations for the verifier (or $O(k^2)$ oracle queries). SafetyNets [9] and zkCNN [14] use this approach. Unlike sumcheck-based methods, our proximity-based checks are purely combinatorial: they compare codeword coordinates via inner products rather than evaluating polynomial extensions, avoiding the algebraic overhead of sumcheck rounds. Moreover, our protocol naturally interfaces with any SNARK backend (the local checks are expressed as R1CS or PlonK constraints), whereas sumcheck-based systems typically require a specialized verifier or a dedicated GKR-to-SNARK compilation step.

8 Conclusion and Future Work

We have presented FastMatMul, a verifiable computation protocol for matrix multiplication whose matrix-checking circuit has size $O(tk + t \log n + E_{cc}(k))$, where t is a small constant determined by the security parameter and $E_{cc}(k)$ is the cost of the selected code-commitment backend relation. With a linear-time code-commitment backend, this is $O(tk)$ in the matrix dimension. The protocol combines Freivalds’ randomized reduction with proximity testing over linear error-correcting codes, cleanly decoupling the expensive matrix arithmetic (performed outside the SNARK by the prover) from the lightweight algebraic checks (arithmetized inside the SNARK). We proved that FastMatMul achieves perfect completeness and computational soundness with error $\epsilon_{\text{SNARK}} + \epsilon_{cc} + \epsilon_{\text{bind}} + (k - 1)/|\mathbb{F}| + 4(1 - \delta)^t$ for the interactive protocol, with the standard additional $\epsilon_{\text{FS}}(q_H)$ loss for the Fiat–Shamir variant in the random oracle model.

Our experimental evaluation demonstrates that the checking-layer advantages translate into concrete gains under the conditional linear-time backend model: FastMatMul reduces the constraint count by 7.5 $\times$ at $k = 1,024$ and 15.1 $\times$ at $k = 2,048$ relative to a direct Freivalds SNARK baseline. At the largest dimension where both systems complete, FastMatMul achieves up to a 2.95 $\times$ proving-time speedup, and it continues to scale at $k = 4,096$ where the baseline exhausts available memory in our benchmark environment. The off-circuit encoding cost is negligible (under 1 s at $k = 2^{20}$), confirming that the code-based approach does not introduce a new bottleneck.

Limitations. The current construction is intentionally stated relative to certified encoded input commitments and a code-commitment

backend; it is not an end-to-end system for producing those commitments from raw inputs, nor does the current artifact isolate an independently measured CodeCom.Verify component. The evaluated Merkle/CPLink-style backends have proof sizes in the megabyte range because sampled-column opening and linking material is explicit. The current artifact does not log peak memory or repeated-run variance, so the out-of-memory baseline result is reported as an observed benchmark outcome rather than a memory-scaling model. The protocol as presented is verifiable computation rather than a full zero-knowledge construction, and the experimental comparison is currently limited to an optimized Freivalds SNARK baseline.

Future directions. Several avenues remain. First, while our protocol targets a single matrix multiplication, deep learning inference involves *chains* of matrix multiplications interleaved with non-linear activations. Extending FastMatMul to handle layered computations—by chaining the commitment outputs of one layer as inputs to the next—is a natural step toward fully verifiable neural network inference. Second, dense backend codeword-validity checks, which contribute $O(k^2)$ under dense general constant-rate codes, could potentially be replaced by a separate proximity test (e.g., FRI [2]) or a linear-time code-commitment backend, further reducing the circuit. Third, replacing the Merkle-based commitment with an algebraic vector commitment (e.g., KZG) would reduce proof sizes from megabytes to kilobytes at the cost of a structured setup; evaluating this trade-off in practice is an important engineering task. Finally, exploring *batched verification*—where multiple matrix multiplications share a single encoding and commitment phase—could yield significant amortized cost savings in practical deployments.

References

- [1] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. 2017. Liger: Lightweight Sublinear Arguments Without a Trusted Setup. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, New York, NY, USA, 2087–2104.
- [2] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 14:1–14:17.
- [3] Huck Bennett, Karthik Gajulapalli, Alexander Golovnev, and Evelyn Warton. 2023. Matrix multiplication verification using coding theory. *arXiv preprint arXiv:2309.16176* (2023).
- [4] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. Zkml: An optimizing system for ml inference in zero-knowledge proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems*. ACM, New York, NY, USA, 560–574.
- [5] Mingshu Cong, Tsz Hon Yuen, and Siu-Ming Yiu. 2024. zkmatrix: Batched short proof for committed matrix multiplication. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. ACM, New York, NY, USA, 289–305.
- [6] Mingshu Cong, Tsz Hon Yuen, and Siu Ming Yiu. 2026. Dualmatrix: conquering zkSNARK for large matrix multiplication. *Cybersecurity* 9, 1 (2026), 126.
- [7] Amos Fiat and Adi Shamir. 1987. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology – CRYPTO ’86*. Springer, Berlin, Heidelberg, 186–194.
- [8] Rūsinš Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. In *Information Processing 77 (Proceedings of IFIP Congress 77)*. North-Holland, Amsterdam, The Netherlands, 839–842.
- [9] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. 2017. Safetynets: Verifiable execution of deep neural networks on an untrusted cloud. *Advances in Neural Information Processing Systems* 30 (2017).
- [10] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2015. Delegating computation: Interactive proofs for muggles. *J. ACM* 62, 4 (2015), 1–64.
- [11] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. 2023. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In *Advances in Cryptology – CRYPTO 2023*. Springer, Cham, 193–226.
- [12] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *Advances in Cryptology – ASIACRYPT 2010*. Springer, Berlin, Heidelberg, 177–194.
- [13] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vcn: Verifiable convolutional neural network based on zk-snarks. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270.
- [14] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. ACM, New York, NY, USA, 2968–2985.
- [15] Ralph C. Merkle. 1989. A Certified Digital Signature. In *Advances in Cryptology – CRYPTO ’89*. Springer, Berlin, Heidelberg, 218–238.
- [16] Torben Prids Pedersen. 1991. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *Advances in Cryptology – CRYPTO ’91*. Springer, Berlin, Heidelberg, 129–140.
- [17] Jacob T. Schwartz. 1980. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM* 27, 4 (1980), 701–717.
- [18] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Berkeley, CA, USA, 501–518.
- [19] Tiancheng Xie, Yupeng Zhang, and Dawn Song. 2022. Orion: Zero Knowledge Proof with Linear Prover Time. In *Advances in Cryptology – CRYPTO 2022*. Springer, Cham, 299–328.
- [20] Yancheng Zhang, Mengxin Zheng, Xun Chen, Jingtong Hu, Weidong Shi, Lei Ju, Yan Solihin, and Qian Lou. 2025. zkvc: Fast zero-knowledge proof for private and verifiable computing. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, New York, NY, USA, 1–7.
- [21] Richard Zippel. 1979. Probabilistic algorithms for sparse polynomials. In *Symbolic and Algebraic Computation (EUROSAM ’79)*. Springer, Berlin, Heidelberg, 216–226.

A Probabilistic Verification of Reed–Solomon Codewords

Let $\mathbb{F}$ be a finite field, let $\mathbf{m} = (a_0, \dots, a_{k-1}) \in \mathbb{F}^k$, and let $\mathbf{c} = (c_0, \dots, c_{n-1}) \in \mathbb{F}^n$ for $n \geq k$. Let $S = \{w^0, w^1, \dots, w^{n-1}\} \subseteq \mathbb{F}$ be the evaluation domain. Define the message polynomial $f(X) = \sum_{i=0}^{k-1} a_i X^i$ and let $g(X)$ be the unique polynomial of degree less than n satisfying $g(w^i) = c_i$ for every i . The goal is to test whether $f(X) \equiv g(X)$, i.e., whether $\mathbf{c}$ is the Reed–Solomon encoding of $\mathbf{m}$.

The verifier samples $z \leftarrow \mathbb{F} \setminus S$, computes $v_1 = f(z)$ by Horner evaluation, and computes $v_2 = g(z)$ by barycentric interpolation:

$$v_2 = \frac{\sum_{i=0}^{n-1} \frac{\lambda_i}{z - w^i} c_i}{\sum_{i=0}^{n-1} \frac{\lambda_i}{z - w^i}},$$

where λ_i are the barycentric weights for S . The verifier accepts iff $v_1 = v_2$.

If $f(X) \not\equiv g(X)$, then $h(X) = f(X) - g(X)$ is a nonzero polynomial of degree less than n . By Schwartz–Zippel,

$$\Pr_{z \leftarrow \mathbb{F}} [f(z) = g(z)] \leq \frac{n-1}{|\mathbb{F}|},$$

which is negligible when $|\mathbb{F}|$ is sufficiently large.